ALGORITMIA

Grado en Ingeniería Informática del Software - Curso 2025-2026

GUION DE LA PRÁCTICA 6

Vuelta Atrás – Backtracking

Se considera el **problema de Bin Packing** (Garey, M. R., & Johnson, D. S., 1979) en su versión clásica: dado un conjunto finito de objetos, cada uno con un tamaño positivo, y un número ilimitado de contenedores idénticos (bins), cada uno con una **capacidad máxima fija**, se desea determinar la asignación de objetos a contenedores que **minimiza el número total de contenedores utilizados**, cumpliendo en todos los casos la restricción de capacidad.

Este problema está reconocido como **NP-hard**, por lo que no se conocen algoritmos de tiempo polinómico que garanticen la obtención de la solución óptima para instancias generales.

El objetivo del ejercicio es diseñar e implementar un algoritmo **exacto** basado en **backtracking (¿con poda?)**, capaz de explorar el espacio de soluciones y determinar el número mínimo de contenedores necesarios para almacenar todos los objetos de una instancia dada.

La solución deberá, como mínimo:

1. Explorar recursivamente todas las posibles asignaciones de objetos a contenedores.
2. Mantener y actualizar en todo momento la **mejor solución encontrada**.
3. Conservar una representación del estado que permita realizar operaciones de avance y retroceso de manera eficiente.

ESPECIFICACIÓN FORMAL DEL PROBLEMA

Sean:

- $S = \{s_1, s_2, \dots, s_n\}$ un conjunto de objetos, donde cada objeto S_i tiene un tamaño entero positivo.
- $C \in \mathbb{N}$ la capacidad entera fija de cada contenedor.
- Conjuntos $B_1, B_2, \dots, B_k$ representando los contenedores utilizados, donde cada $B_j \subseteq S$.

El objetivo es encontrar el mínimo valor k tal que:

- Ningún contenedor exceda la capacidad máxima permitida:

$$\sum_{s \in B_j} s \leq C \; \forall j \in \{1, \dots, k\}$$

- La unión de todos los contenedores contiene exactamente el conjunto de todos los objetos y los contenedores forman una partición del conjunto de objetos:

$$\bigcup_{j=1}^{k} B_j = S, B_i \cap B_j = \emptyset \text{ si } i \neq j.$$

ENTRADA Y SALIDA DEL PROGRAMA

Entrada:

Un archivo de texto con el siguiente formato:

- Primera línea: un entero positivo C, que es la capacidad de los contenedores.
- Segunda línea: una secuencia de enteros positivos que representan los tamaños de los objetos.

Por ejemplo, en **test00.txt** se proporciona la siguiente entrada:

```
10
7 6 5 5 4 3 2
```

Salida:

El programa debe producir:

1. El número mínimo de contenedores necesarios para almacenar todos los objetos.
2. La distribución final de los objetos en cada contenedor, enumerados desde 1 hasta el número total de contenedores utilizados.

Por ejemplo, para la entrada dada anterior una solución puede ser la siguiente:

```
Lista de contenedores y objetos contenidos:
     Contenedor 1: 3 7
     Contenedor 2: 4 6
     Contenedor 3: 5 5
     Contenedor 4: 2

El número de contenedores necesario es 4.
```

3. Opcionalmente también se puede imprimir el número de llamadas recursivas realizadas.

Se le pide:

A) Una clase **p6.AlmacenajeContenedores.java**, que tras ejecutarse así:

java p6.AlmacenajeContenedores fichero.txt

Solucione el problema tal y como se indicó.

B) Una clase **p6.AlmacenajeContenedoresTiempos.java**, que tras ejecutarse así:

java p6.AlmacenajeContenedoresTiempos

Ejecutará todos los ficheros de prueba para calcular el tiempo necesario para resolver el problema. Si para algún el programa tarda mucho, se podrá detener para ese caso concreto.

C) Para finalizar, rellene una tabla en la que aparezcan para los ficheros antes indicados los tiempos obtenidos (en milisegundos).

¿Qué complejidad asigna al algoritmo que calcula el número mínimo de contenedores a utilizar? ¿Por qué?

Se ha de entregar en un **.pdf** el trabajo que se le pide y además las clases **.java** que ha programado. Todo ello lo organizará en una carpeta, que es la que entregará comprimida en un fichero **p6ApellidosNombre.zip**.

La entrega de esta práctica se realizará, en tiempo y forma, según las indicaciones dadas por el profesor de práctica.